

DPDK Encrypted Tunnel

Complete Technical Guide

Control Plane · Data Plane · Key Management · Security Features · Architecture

DPDK Version	22.11.6 LTS
Target Hardware	NXP LS-series (DPAA2) + x86 RHEL
Cipher	AES-128-CBC (CIPHER_ONLY chain)
Key Exchange	TLS 1.3 + RFC 5705 Exporter
Control Path	virtio-user / vhost-net (ctrl10)
Document Date	June 25, 2026

Table of Contents

1. System Architecture Overview
2. Control Plane — Virtual Port Design
3. How Ping Works Through the Virtual Port
4. TLS and OpenSSL — What Each Does
5. Key Management — PSK to TLS-Derived Keys
6. Key Flow to DPDK Data Plane
7. Data Plane — Encrypt / Decrypt Pipeline
8. RX / TX Queue Formation
9. MTU Fix — Root Cause and Solution
10. Fragmentation and Reassembly
11. Security Features
12. Dead Node Detection
13. Replay Attack Prevention
14. Alternative Architectures
15. IPsec Feature Gap Analysis
16. NIC Driver — What It Is
17. Bare Metal TLS — Is It Possible
18. Development Timeline

1. System Architecture Overview

The system implements an encrypted Layer 2 tunnel between two DPDK-enabled machines. Each machine has a client or server connected to its LAN-facing port. The DPDK application encrypts all traffic leaving the LAN port before it reaches the WAN wire, and decrypts all traffic arriving from the WAN wire before forwarding to the LAN-side server.

Client	→	DPDK Box 1 (Encrypt)	→ WAN wire →	DPDK Box 2 (Decrypt)	→	Server
192.168.x.x		Port 0: LAN Port 1: WAN		Port 1: WAN Port 0: LAN		10.x.x.x

Two Planes — Why They Are Separated

The system is divided into two completely separate planes:

- **Data Plane:** Handles actual user traffic — encrypt/decrypt at line rate using DPDK and CAAM hardware. Runs entirely in user space, bypasses Linux kernel for NIC access.
- **Control Plane:** Handles key exchange (TLS 1.3), key rotation, and management. Runs keyd as a kernel process using standard Linux networking.

Separating these planes means a control plane failure (keyd crash, TLS reconnect) does not interrupt data forwarding. The data plane continues with the last known key until the control plane recovers.

2. Control Plane — Virtual Port Design

The fundamental problem: DPDK binds the physical NIC exclusively using `vfiopci/uiso`. Once bound, the Linux kernel's NIC driver is detached — the kernel cannot send or receive anything through that NIC. `keyd` runs in user space and uses kernel TCP/IP sockets. Without intervention, `keyd` has no path to the peer box's `keyd` over the DPDK-bound WAN link.

The Virtual Port Solution

The `--vdev=virtio_user0,path=/dev/vhost-net,iface=ctrl10,queues=1` argument creates a virtual NIC with two halves:

- **Kernel half (`ctrl10`):** A TAP-like interface visible to the Linux kernel. The kernel assigns it an IP (169.254.0.1), and `keyd` uses it exactly like any normal NIC.
- **DPDK half (`virtio_user0`):** A DPDK `ethdev` port. The DPDK app reads frames from it and forwards them to the physical WAN port.
- **Bridge — `/dev/vhost-net`:** A kernel module that uses shared memory rings to pass frames between the two halves with zero data copy and no syscall overhead.

`ctrl_bridge.c` — The Glue Code

The `ctrl_bridge` module (`ctrl_bridge.c` / `ctrl_bridge.h`) manages this bridge:

- **`cb_init()`:** Finds the `virtio_user0` port, configures it, starts it, and sets the kernel-side MTU.
- **`cb_pump_egress()`:** Called every main loop iteration. Drains frames from the kernel (`virtio_user0` RX), tags them with EtherType 0x88B5, and TX them to the WAN port.
- **`cb_is_ctrl_frame()`:** Checks if a WAN RX frame has EtherType 0x88B5 — one comparison.
- **`cb_punt_to_kernel()`:** Strips the 2-byte tag, restores original EtherType, and TX to `virtio_user0` so the kernel receives a clean frame on `ctrl10`.

EtherType Tagging — Why It Is Needed

Both control frames (ARP, ICMP, TLS) and encrypted data frames share the same physical WAN wire. The DPDK app must distinguish them. The solution: tag every control-plane frame with EtherType 0x88B5 (IEEE-assigned for local experimental use) on egress, and detect this tag on ingress to divert to the kernel instead of the decrypt pipeline.

```
Wire layout (control frame):
```

```
[ETH dst|src|0x88B5][orig EtherType 2B][original L3 payload...]
```

```
Wire layout (data frame):
```

```
[ETH dst|src|0x0800][Outer-IP][UDP][IV][Encrypted(Inner-IP+TCP+Data)]
```

3. How Ping Works Through the Virtual Port

This traces exactly what happens when 'ping 169.254.0.2' is run on the RHEL machine with 169.254.0.1 on ctrl10.

Step 1 — Kernel routing decision

Kernel checks routing table. Finds: 169.254.0.0/16 dev ctrl10. Decides to send via ctrl10.

Step 2 — ARP request

Kernel has no ARP entry for 169.254.0.2. Builds broadcast ARP request (EtherType 0x0806, dst MAC ff:ff:ff:ff:ff:ff).

Step 3 — Frame into DPDK via vhost-net

Kernel writes ARP frame to ctrl10 TAP driver. TAP places frame in shared vhost-net ring. DPDK virtio_user0 now has it in its RX ring.

Step 4 — cb_pump_egress() picks it up

DPDK main loop calls cb_pump_egress(). RX burst from virtio_user0. Saves original EtherType 0x0806. Prepends 2-byte tag. Sets outer EtherType to 0x88B5. TX out physical WAN port.

Step 5 — Frame crosses the wire

NXP DPAA2 hardware DMA's the tagged frame onto the physical cable.

Step 6 — NXP receives on WAN port

NXP main loop RX burst on port 1 receives the frame.

Step 7 — cb_is_ctrl_frame() identifies it

EtherType == 0x88B5 check returns true. Calls cb_punt_to_kernel(m).

Step 8 — Frame restored and punted to kernel

Reads 2-byte tag (0x0806). Strips 2 bytes. Restores original ETH header with EtherType 0x0806. TX to NXP virtio_user0. Kernel receives clean ARP on ctrl10.

Step 9 — NXP kernel responds

NXP kernel ARP handler: 'I have 169.254.0.2'. Builds ARP reply. Reply travels back via same path in reverse.

Step 10 — RHEL kernel updates ARP cache

ARP reply arrives on RHEL ctrl10. Kernel: 169.254.0.2 = NXP ctrl10 MAC.

Step 11 — ICMP Echo follows same path

With ARP resolved, kernel builds ICMP Echo Request. Follows identical path. NXP kernel responds with ICMP Echo Reply. ping prints the RTT.

4. TLS and OpenSSL — What Each Does

TLS (Transport Layer Security) is a protocol — a set of rules defined in RFCs describing how two parties authenticate, exchange keys, and encrypt data. It is just a document. OpenSSL is a library — an implementation of that protocol in C code.

What OpenSSL Provides

OpenSSL Component	What It Does
X.509 certificate parsing	ASN.1/DER format parsing — complex binary structure with nested fields for subject, issuer, validity, public key, extensions. ~5000 lines.
Certificate chain verification	Validates signature chain from leaf cert up to trusted CA. Checks validity dates, key usage extensions.
ECDHE key exchange	Elliptic curve Diffie-Hellman Ephemeral. Generates ephemeral keypair, computes shared secret via point multiplication on P-256 or X25519.
HKDF key derivation	RFC 5869 HKDF-Extract + HKDF-Expand. Derives handshake secrets, traffic secrets, and the exporter master secret from the ECDHE shared secret.
TLS 1.3 state machine	Full handshake: ClientHello, ServerHello, EncryptedExtensions, Certificate, CertificateVerify, Finished — both active and passive sides.
AES-GCM record encryption	Encrypts TLS application data records with authenticated encryption.
RFC 5705 key exporter	<code>SSL_export_keying_material()</code> — derives application-specific keying material from the TLS session's exporter master secret.
Alert handling	TLS alert protocol — signals errors and warnings to the peer.

RFC 5705 Key Exporter — Your Core Mechanism

This is the most important OpenSSL feature in your system. Both keyd instances call:

```
SSL_export_keying_material(ssl, out_key, 16,  
"EXPORTER-l2fwd-traffic-key", 26,  
&epoch;_byte, 1, 1);
```

Because both sides share the same TLS master secret from the handshake, they derive bit-identical 16-byte output independently. No key material ever travels on the wire — it is mathematically derived from the shared TLS secret. Each epoch uses a different context (the epoch number), so each epoch's key is cryptographically independent.

5. Key Management — PSK to TLS-Derived Keys

Bootstrap Phase — PSK

At startup, before `keyd` connects, the data plane needs a key to operate. `key_mgmt.c` loads a 16-byte Pre-Shared Key from `/etc/l2fwd/psk.key` (mode 0600, same file on both boxes). It derives epoch 0's traffic key:

```
K(0) = HKDF(PSK, 'l2fwd epoch 0')
```

```
K(n+1) = HKDF(K(n), 'l2fwd epoch n+1')
```

This ratchet is deterministic — both boxes compute identical keys from the same PSK without any communication. The data link works immediately at startup.

Rotation Phase — TLS-Derived Keys

Once `keyd` establishes TLS between the two boxes, it exports a fresh AES-128 key every `--rotate` seconds (default 120s) via RFC 5705. It pushes this key to the local DPDK app over an `AF_UNIX SOCK_SEQPACKET` socket using the `keyd_proto` message format.

Overlap Window

When a rotation happens, the previous epoch's session is kept alive for `KM_DEFAULT_OVERLAP_SEC` (10 seconds). Every packet carries a 1-byte epoch ID in the outer-IP TOS byte. The receiver uses this to select the correct session — packets encrypted with the old key still decrypt correctly during the overlap window.

6. Key Flow to DPDK Data Plane

The complete path from TLS session to DPDK crypto session:

1. keyd: TLS 1.3 handshake completes between Box 1 and Box 2
2. keyd: `SSL_export_keying_material()` — both sides derive identical 16-byte AES key
3. keyd: Sends `keyd_msg{type=INSTALL_KEY, epoch=N, key[16]}` over `AF_UNIX` socket
4. `key_listener.c`: `kl_poll()` receives message via non-blocking `recv()`
5. `key_listener.c`: Calls `install_cb` (`kr_install_cb` in `main.c`)
6. `kr_install_cb`: Calls `cs_rotate()` for each crypto device on this lcore
7. `crypto_session.c`: `cs_rotate()` builds new `rte_cryptodev_sym_session` with new key
8. `crypto_session.c`: Moves current session to 'prev' slot (overlap window)
9. Data plane: `cs_session_for_epoch()` selects session matching packet's TOS epoch byte
10. CAAM hardware: Uses session's baked-in key to encrypt/decrypt the packet

Per-Epoch Session Table (`crypto_session.c`)

A DPDK crypto session bakes the cipher key in at creation time — the key cannot be mutated on a live session. `cs_port` keeps two session slots: current epoch and previous epoch. `cs_rotate()` builds a fresh session with the new key and keeps the old one for the overlap window. `cs_retire_prev()` frees the old session after the window closes.

7. Data Plane — Encrypt / Decrypt Pipeline

Wire Format

Every encrypted packet on the WAN wire has this layout:

```
[ETH 14B][Outer-IP 20B][UDP 8B][IV 16B][Encrypted(Inner-IP+TCP/UDP+Data+Padding)]
```

Total fixed overhead: 58 bytes per packet. Inner packet is fully encrypted including the inner IP header — providing confidentiality of traffic endpoints.

Encrypt Path (Port 0 — Client Side)

1. RX burst from port 0 — client's packet arrives
2. ARP check: if ARP, forward directly without crypto
3. `tf_reassemble_rx()`: collect inbound client fragments if any
4. `rg_stamp()`: prepend 8-byte replay guard header (timestamp + seq)
5. `tf_fragment_inner()`: split if inner packet + 58B overhead > WAN MTU
6. `l2fwd_simple_crypto_enqueue()`: generate fresh IV, attach crypto session, enqueue to CAAM
7. CAAM hardware encrypts: AES-128-CBC from byte 14 (inner IP) onward
8. Dequeue completed op from CAAM
9. `tunnel_encap()`: prepend [Outer-IP][UDP][IV], stamp epoch in TOS byte
10. `l2fwd_send_packet()` → TX burst out port 1 (WAN)

Decrypt Path (Port 1 — WAN Side)

1. RX burst from port 1 — encrypted frame arrives from peer
2. `cb_is_ctrl_frame()`: if EtherType 0x88B5, punt to kernel — skip decrypt
3. `tunnel_decap()`: strip [Outer-IP][UDP], read epoch from TOS, extract IV
4. Epoch check: `cs_session_for_epoch()` — drop if epoch unknown
5. `l2fwd_simple_crypto_enqueue()`: attach session for this epoch, enqueue to CAAM
6. CAAM hardware decrypts: AES-128-CBC
7. `ic_verify_digest()`: verify HMAC/GCM tag if auth chain used
8. `tf_reassemble_decap()`: reassemble inner fragments if this was a split packet
9. `rg_verify()`: check timestamp freshness + sequence number replay window
10. Trim AES-CBC padding using inner-IP `total_length` field
11. `l2fwd_send_packet()` → TX burst out port 0 (server side)

8. RX / TX Queue Formation

DPDK uses descriptor ring buffers for NIC communication. The NIC hardware and CPU share memory-mapped rings. The CPU writes packet addresses into TX descriptors and kicks the hardware register. The NIC DMAs data from RAM onto the wire without CPU involvement.

Ring Configuration

Parameter	Value	Meaning
RX descriptors per port	1024 (nb_rxd)	Packets the NIC can buffer before CPU drains
TX descriptors per port	1024 (nb_txd)	Packets CPU can queue before NIC drains
RX burst size	MAX_PKT_BURST = 32	Packets pulled per <code>rte_eth_rx_burst()</code> call
Crypto op burst	MAX_PKT_BURST = 32	Ops pushed to CAAM per enqueue call
Drain interval	100 microseconds	BURST_TX_DRAIN_US — force flush every 100us
Lcores	One per port	Lcore 0 = port 0 encrypt, Lcore 1 = port 1 decrypt

Packet Flow Through Queues

```
NIC RX ring → rte_eth_rx_burst() → pkts_burst[32]
→ l2fwd_simple_crypto_enqueue() → op_buf (software)
→ rte_cryptodev_enqueue_burst() → CAAM HW queue
→ rte_cryptodev_dequeue_burst() → completed ops
→ l2fwd_simple_forward() → pkt_buf (software)
→ rte_eth_tx_burst() → NIC TX ring → wire
```

The drain timer (100us) ensures packets never sit in the software buffers longer than 100 microseconds even if the burst never fills to 32.

9. MTU Fix — Root Cause and Solution

The Bug

ctrl10 defaulted to MTU 1500. `cb_pump_egress()` prepends 2 bytes to every frame (the original EtherType tag). A 1500-byte kernel frame becomes 1502 bytes after tagging — exceeding what the WAN port/crypto pipeline can carry.

Small packets (SYN/ACK, ping) stayed under 1500 even after tagging. The TLS certificate message (a 1448-byte TCP segment + tagging = over 1500) was silently dropped. `SSL_connect()` hung indefinitely waiting for the certificate that never arrived.

Why It Was Hard to Find

- Ping worked — ICMP is small
- Raw TCP (nc) worked — test messages are small
- TLS SYN/ACK/ClientHello worked — small packets
- Only the certificate-bearing TLS record failed — large enough to overflow after tagging

The Fix — Two Layers

Layer 1 — DPDK side (before queue setup):

```
ret = rte_eth_dev_set_mtu(pid, RTE_ETHER_MTU - CB_TAG_OVERHEAD);  
// RTE_ETHER_MTU (1500) - CB_TAG_OVERHEAD (2) = 1498
```

Layer 2 — Kernel side (after `dev_start`, because virtio-user PMD does not propagate `set_mtu` to the kernel interface):

```
snprintf(cmd, sizeof(cmd), "ip link set %s mtu %u",  
CB_IFACE_NAME, RTE_ETHER_MTU - CB_TAG_OVERHEAD);  
system(cmd); // sets ctrl10 MTU to 1498 in kernel
```

With MTU 1498, TCP's MSS negotiation caps segments so they never produce a frame larger than 1498 bytes. After the 2-byte tag: $1498 + 2 = 1500$. Exactly at the limit, never over. TLS handshake completes cleanly.

10. Fragmentation and Reassembly

The `tunnel_frag` module handles two scenarios where packets need fragmenting or reassembling.

Scenario 1 — Packet Grows After Encapsulation

`tunnel_encap()` adds 58 bytes of overhead (Outer-IP 20 + UDP 8 + IV 16 + ETH 14) to every packet. A full 1500-byte client packet becomes 1558 bytes after encapsulation — 58 bytes over Ethernet MTU. Solution: fragment the inner packet BEFORE encryption.

```
tf_fragment_inner(m, frags) // splits if inner + 58B > 1500
// Each fragment individually encrypted with unique IV
// Peer: decrypt each → tf_reassemble_decap() → complete packet
```

Scenario 2 — Client Sends Fragmented Packets

A client upstream of the DPDK box may send IP-fragmented packets. These must be reassembled into a complete inner packet BEFORE encryption. Otherwise each fragment gets its own tunnel header and the peer cannot reconstruct the original.

```
tf_reassemble_rx(m, cur_tsc) // reassemble before encrypt
```

Key Design Decisions

Decision	Reason
Two separate tables	One for inbound client fragments (port 0 RX), one for decap'd fragments (port 1 RX). Different flows, must not mix.
IV uniqueness	Each fragment gets its own unique IV via <code>generate_per_pkt_iv()</code> . IV reuse under same key is a cryptographic weakness for AES-CBC.
<code>tf_drain_death_row()</code>	Called every drain interval. Evicts stale incomplete fragment sequences. Without this, lost fragments permanently occupy table slots — the root cause of the vendor's 3-day failure.
Linearize after reassembly	<code>rte_ipv4_frag_reassemble_packet()</code> returns a multi-segment mbuf. <code>rte_pktmbuf_linearize()</code> makes it contiguous before downstream code uses <code>mtod()</code> .
<code>TF_MAX_ENTRIES = 512</code>	64 buckets x 8 entries. Limits memory use while covering realistic concurrent fragment flows.

11. Security Features

Feature	Description
AES-128-CBC	Data confidentiality. CAAM hardware acceleration. Per-packet random IV prevents identical ciphertext for identical plaintext.
TLS 1.3 key exchange	Mutual certificate authentication. ECDHE forward secrecy. Each session uses fresh ephemeral keys.
RFC 5705 key export	Traffic keys derived from TLS session — never transmitted on wire. Both boxes derive identical keys independently.
HKDF ratchet	PSK bootstrap: $K(n+1) = \text{HKDF}(K(n), \text{'l2fwd epoch } n+1')$. Deterministic, same on both boxes from same PSK.
Epoch-based rotation	1-byte epoch in outer-IP TOS. 120-second rotation interval. 10-second overlap window prevents drops at rotation boundary.
Replay prevention	8-byte header (4B timestamp + 4B sequence) inside encrypted payload. 1024-packet sliding window bitmap. Configurable max age (default 120s).
Dead node detection	keyd sends PEER_DOWN/PEER_UP over AF_UNIX when TLS peer closes. DPDK app halts forwarding until peer recovers.
Rate limiting	Token bucket rate limiter on WAN port. Prevents crypto op pool exhaustion from packet flood attacks.
Volume-based rotation	SA byte-count limit triggers early rotation independent of time interval.
Traffic flow confidentiality	Optional padding to fixed packet size (TFC_PAD_TARGET) to hide traffic patterns from observers.

■ **CRITICAL GAP: AES-CBC without HMAC provides confidentiality but NOT integrity. An attacker can flip bits in ciphertext and corrupt decrypted traffic without detection. Switch to --chain AEAD --aad_algo aes-gcm or --chain CIPHER_HASH --auth_algo sha256-hmac immediately.**

13. Replay Attack Prevention

AES-CBC with per-packet IV prevents decryption of modified packets. It does NOT prevent an attacker from capturing and re-injecting an unmodified packet. The crypto succeeds and the inner packet is forwarded as legitimate traffic.

The `replay_guard` module (`replay_guard.c`) adds a timestamp + sequence number header inside the encrypted payload — unmodifiable without breaking decryption.

Replay Guard Header (8 bytes, inside encrypted payload)

```
struct rg_hdr {  
    uint32_t timestamp_be; // Unix time (seconds), network order  
    uint32_t seq_be; // Monotonic sequence number  
} __attribute__((packed));
```

Verification Logic (`rg_verify`)

- Extract timestamp and seq from decrypted packet header
- Reject if packet older than `RG_MAX_AGE_SEC` (configurable, default 120s)
- Reject if timestamp more than 30s in future (clock skew / replay attack)
- Check 1024-entry sliding window bitmap — reject if seq already seen
- Mark seq as seen in bitmap
- Strip 8-byte header — forward clean inner packet

Log Output	Meaning
[REPLAY] DROPPED: packet too old	age > <code>RG_MAX_AGE_SEC</code> -- stale replayed packet
[REPLAY] DROPPED: seq already seen	Exact duplicate -- captured and retransmitted
[REPLAY] DROPPED: timestamp in future	Clock skew > 30s -- possible attack
[REPLAY] DROPPED: seq too far behind	Sequence behind sliding window -- too old

Note: Requires NTP on both boxes to keep clocks in sync. Clock drift greater than `RG_MAX_AGE_SEC` causes legitimate packets to be dropped.

14. Alternative Architectures

The virtual port (ctrl10/virtio-user) was one solution. These alternatives would have worked without it:

Approach	Complexity	Description
Second physical NIC	Low	NIC 0 → DPDK data plane. NIC 1 → kernel, keyd uses it directly. No virtual port, no ctrl_bridge, no MTU tagging. Best choice if hardware allows.
Management network	Low	keyd uses a completely separate network path (e.g. SSH/management NIC). Data plane on DPDK WAN. Cleanest separation. How real IPsec deployments work.
AF_XDP	Medium	Newer Linux feature. Shares NIC between kernel and DPDK via eBPF filtering. Supported in DPDK 22.11 via net_af_xdp PMD. Modern single-NIC solution.
F-Stack / userspace TCP	High	FreeBSD TCP/IP stack ported to run on DPDK mbufs. keyd uses ff_socket/ff_connect instead of kernel sockets. Eliminates virtual port entirely. IP assigned via config.ini.
KNI	N/A	Deprecated and removed in DPDK 22.11. Would have worked on older versions. Out-of-tree kernel module required rebuilding for each kernel version.

Why Virtual Port Was the Right Choice Here

- Single NIC per box — no second physical NIC available
- KNI removed in DPDK 22.11 — not an option
- F-Stack integration = significant rewrite of keyd + main loop
- Virtual port works correctly with the MTU fix in place
- Management network not available in this lab setup

15. IPsec Feature Gap Analysis

Feature	Status	Notes
AES-128-CBC encryption	✓ Done	DPDK CAAM hardware
TLS 1.3 key exchange	✓ Done	RFC 5705 exporter
Key rotation (time)	✓ Done	120s default interval
Key rotation (volume)	Partial	Scaffolded, needs byte counter wired in
Replay prevention	✓ Done	Timestamp + seq window
Dead node detection	✓ Done	PEER_DOWN/UP via keyd
Tunnel mode	✓ Done	Full inner IP encapsulation
Fragmentation/reassembly	✓ Done	tunnel_frag.c
Control/data plane separation	✓ Done	ctrl_bridge + virtio-user
Integrity / Authentication	CRITICAL MISSING	No HMAC — switch to AEAD/AES-GCM
Anti-clogging / rate limiting	Done	rate_limiter.c token bucket
Perfect forward secrecy	Partial	TLS ECDHE OK; PSK file is long-term static
Traffic flow confidentiality	Done	tfc_pad.c — optional fixed-size padding
ICMP error translation	Missing	PMTUD breaks without this for tunneled clients
Extended sequence numbers	Missing	32-bit seq exhausts in 85min at 10Gbps
PSK elimination (ECDHE)	Scaffolded	km_run_ecdhe_handshake() — not implemented

16. NIC Driver — What It Is

A NIC driver is code that talks directly to NIC hardware registers to make it send and receive packets. DPDK's PMD (Poll Mode Driver) IS a NIC driver — when you call `rte_eth_tx_burst()`, DPDK internally calls the hardware-specific driver function.

Five Jobs of Every NIC Driver

Job	Description
Initialization	Reset hardware, set MAC address, configure flow control, enable RX/TX. Follows a hardware-specific sequence documented in the NIC's datasheet.
DMA descriptor ring setup	Allocate circular ring buffers in DMA-accessible RAM. Tell NIC the physical address of the ring. NIC reads TX descriptors and fills RX descriptors without CPU involvement.
TX path	Write packet address + length into TX descriptor. Advance tail pointer. Write tail to NIC register (the 'kick'). NIC DMAs packet from RAM to wire.
RX path	Pre-post empty buffers. Poll RX descriptors for the Done bit. When set, NIC has written a received packet. Take the buffer, post a fresh one.
Link management	Read link status register to detect cable plug/unplug. Read speed/duplex registers. Handle auto-negotiation.

Why It Is Hardware-Specific

Intel i210, Mellanox ConnectX, and NXP DPAA2 all implement the same Ethernet standard but with completely different register layouts, descriptor formats, and initialization sequences. Intel uses simple ring + tail register. NXP DPAA2 uses QBMAN portals and Frame Descriptors. These are incompatible at the driver level — you cannot use an Intel driver on an NXP NIC or vice versa.

DPDK's PMD layer hides this behind a common API: `rte_eth_tx_burst()`, `rte_eth_rx_burst()`, `rte_eth_dev_configure()`. Your code calls these functions and DPDK dispatches to the correct hardware-specific driver for your NIC.

17. Bare Metal TLS — Is It Possible

Yes. TLS on bare metal without an OS is done by embedded security devices worldwide — routers, HSMS, VPN appliances, smart cards. The key insight: TLS libraries like mbedTLS and wolfSSL have zero OS dependency. They need only two function pointers: send and recv.

What TLS Needs from the 'OS'

Requirement	Why TLS Needs It
Reliable ordered byte stream	TCP state machine — SYN/ACK/FIN, retransmission, flow control, congestion control. TLS has no retransmission of its own.
IP routing	Which interface to use for a given destination IP. ARP to resolve IP to MAC.
Port binding	Bind to port 4434, listen, accept incoming connections. Multiplexes multiple applications on one IP.
File descriptor abstraction	OpenSSL uses read()/write() on socket fd. Needs something to wrap the transport.

Stack for Bare Metal TLS

```
Hardware NIC registers
↓ NIC driver (~500-2000 lines, hardware-specific)
↓ Ethernet framing (~100 lines)
↓ ARP + IP (~500 lines, or use lwIP)
↓ TCP (~500+ lines, or use lwIP)
↓ Two callbacks: send(buf, len) + recv(buf, len)
↓ mbedTLS or wolfSSL (just point at your callbacks)
↓ TLS 1.3 handshake + record encryption
```

Relevance to Your DPDK System

Your DPDK system is already very close to bare metal. DPDK PMD = NIC driver. rte_ether/rte_ip = Ethernet/IP helpers. What is missing for full bare metal TLS is just TCP (for key's connection) and pointing mbedTLS/wolfSSL at DPDK mbufs instead of kernel sockets. This is exactly what F-Stack does — packaged cleanly.

18. Development Timeline

Estimated effort for remaining work, assuming one developer familiar with the codebase:

Feature	Effort	Notes
Integrity / AES-GCM switch	1 day	CRITICAL — do first. Change --chain to AEAD.
Rate limiting	1 day	Code written (rate_limiter.c). Wire into main.c.
Volume-based SA rotation	1 day	Add byte counter, km_force_rotate().
Traffic flow confidentiality	1 day	Code written (tfc_pad.c). Integration + test.
Extended 64-bit sequence numbers	1 day	Mechanical change in replay_guard.h/c.
ICMP error translation	3-4 days	Parse ICMP type 3/11 on WAN, translate to inner.
PSK elimination / ECDHE handshake	5-7 days	km_run_ecdhe_handshake() implementation.
F-Stack integration (optional)	2-3 weeks	Eliminate virtual port. Rewrite keyd I/O layer.
Total (without F-Stack)	13-16 days	Production-quality IPsec feature set.

Recommended Order

- **Week 1:** Integrity (GCM) + Rate limiting + Volume rotation + TFC + Extended seq. All straightforward, all high value. End of week = production-quality security baseline.
- **Week 2-3:** ICMP error handling. Complex but important for real deployment PMTUD.
- **Week 3-4:** ECDHE / PSK elimination. Touches key management core — test carefully.
- **Optional later:** F-Stack integration. Only justified for production single-NIC deployment where virtual port complexity is unacceptable.

Appendix — Kernel Space vs User Space

Every piece of software on a Linux system runs in one of two worlds:

Kernel Space

- Where the Linux OS itself runs
- Direct access to hardware — can write to NIC registers, manage memory, schedule processes
- A crash here crashes the entire machine (kernel panic)
- Runs in CPU ring 0 (highest privilege)
- Contains: TCP/IP stack, NIC drivers, file system, process scheduler

User Space

- Where every normal program runs (keyd, sshd, your browser)
- Cannot directly touch hardware — must ask the kernel via syscalls (send(), recv(), open())
- Memory protected — each process only sees its own memory
- A crash kills only that process — everything else keeps running
- Runs in CPU ring 3 (lowest privilege)

Why DPDK Breaks Normal Networking

Normal networking: Application → socket() syscall → kernel TCP/IP → kernel NIC driver → wire

DPDK: Application → DPDK PMD (user space driver) → NIC hardware directly (kernel bypassed)

When DPDK binds a NIC with vfio-pci, the kernel NIC driver is detached. The kernel cannot send or receive anything through that NIC. keyd's kernel sockets have no path to the wire. This is why the virtual port (ctrl10) was needed — to give the kernel a virtual NIC it can drive, while DPDK handles the real physical NIC.

The vhost-net Bridge

vhost-net uses shared memory rings — both kernel and DPDK read/write the same memory region. No data is copied. No syscalls are needed for data transfer. The kernel signals DPDK when data is ready via an eventfd. This is why it is called zero-copy and why control-plane latency through the virtual port is sub-millisecond.